软件学报 ISSN 1000-9825, CODEN RUXUEW
Journal of Software, [doi: 10.13328/j.cnki.jos.000000]
©中国科学院软件研究所版权所有.
E-mail: jos@iscas.ac.cn
http://www.jos.org.cn
Tel: +86-10-62562563

基于 eBPF 的多智能体运行时安全监测与根因定位方法*

唐浩钺†, 刘周康†, 任怡, 张建锋, 赵欣, 马俊, 余杰

(国防科技大学 计算机学院, 湖南 长沙 410073)

通讯作者: 任怡, E-mail: renyi@nudt.edu.cn

摘 要: 大语言模型（LLM）驱动的多智能体系统通过任务委派、上下文交接和子智能体创建组织复杂工作流。攻击载荷可由用户提示词迁移至文件、命令输出等数据通道，其执行行为还可能分散在多个智能体、进程和资源访问关系中，使依赖文本或单智能体上下文的防御难以重建完整的攻击传播路径及其运行时影响。针对上述问题，提出基于 eBPF 的多智能体运行时安全监测与根因定位方法 MtAgentLens。该方法无需修改智能体业务源码，通过采集 Linux 环境中的进程、文件、网络和 SSL/TLS 行为，以策略边界穿越（PBC）量化敏感资源访问和跨智能体委派行为，并结合责任图与六维行为评分完成攻击检测和因果链回溯。为系统评估载荷跨通道迁移和多智能体传播条件下的检测能力，进一步设计 SAFI 攻击场景构造框架，从审查范围迁移、语义包装和跨智能体传播 3 个维度形成 6 种攻击模式，并构建包含 58 个攻击场景和 35 个良性场景的 FuxiBench v2.0 冻结基准。实验结果显示，在 29 个实际执行攻击上，MtAgentLens 的 PP-TPR 为 0.793（23/29），F1 为 0.852，二者均高于全部基线；在 35 个良性场景上，FPR 为 0.057（2/35, 95% CI [0.016, 0.186]），高于完整应用轨迹基线 R-Judge 的 0.029，但仍低于三种文本基线。消融显示，责任图阶段主要提升排序（AUROC 0.590→0.677，95%CI 跨 0），完整判定流程主要改善误报控制与 F1（PP FPR 0.743→0.057，F1 0.690→0.852）。四个代表性案例展示了责任链从告警事件回溯至上游入口的过程；在 16 个多智能体攻击场景上，根因主体定位的 RootAgent@1、RootAgent@2 与 MRR 分别为 0.688、0.875、0.823。开销方面，监测 4 个智能体的常规负载下，系统引入的额外 CPU 占用率为 3.8%，平均 PSS(Proportional Set Size，用于衡量实际使用的物理内存)为 115.7MiB；扩展到监测 32 个智能体并发任务时，仍无受控与持久化缺失，完成率保持 100%。上述结果说明，MtAgentLens 适用于产生操作系统级可观测行为的 Linux 多智能体工作流，可与文本和应用层防御形成互补，并提供基于运行时行为证据的安全监测与根因定位能力。

关键词: 大语言模型；多智能体系统；eBPF；安全监测；根因定位；策略边界穿越；操作系统级可观测性

中图法分类号: TP311

中文引用格式: 唐浩钺, 刘周康, 任怡, 张建锋, 赵欣, 马俊, 余杰. MtAgentLens: 基于 eBPF 的多智能体运行时安全监测与根因定位方法. 软件学报,2026,32(7). http://www.jos.org.cn/1000-9825/0000.htm

英文引用格式: Tang HY, Liu ZK, Ren Y, Zhang JF, Zhao X, Ma J, Yu J. MtAgentLens: An eBPF-Based Method for Runtime Security Monitoring and Root-Cause Localization in LLM-Driven Multi-Agent Systems. Ruan Jian Xue Bao/Journal of Software, 2026 (in Chinese). http://www.jos.org.cn/1000-9825/0000.htm

* 基金项目: 国家重点研发计划(2024YFB4506200); 湖南省联合基金创新研究群体项目(2026JJ10007); 湖南省重点研发计划“面向 AIPC 的智能原生操作系统关键技术研发” 本项目代码已开源：https://github.com/LiuZhouKang/MtAgentLens

† 唐浩钺与刘周康对本研究贡献相同，列为共同第一作者

An eBPF-Based Method for Runtime Security Monitoring and Root-Cause Localization in LLM-Driven Multi-Agent Systems

TANG Hao-Yue†, LIU Zhou-Kang†, REN Yi, ZHANG Jian-Feng, ZHAO Xin, MA Jun, YU Jie

(College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China)

Abstract: Large language model (LLM)-driven multi-agent systems organize complex workflows through task delegation, context handoff, and sub-agent creation. Attack payloads can migrate from user prompts into data channels such as files and command outputs, and their execution may be distributed across agents, processes, and resource accesses. As a result, defenses relying on text or single-agent context may fail to reconstruct the complete attack propagation path and its runtime effects. To address this problem, we propose MtAgentLens, an extended Berkeley Packet Filter (eBPF)-based method for runtime security monitoring and root-cause localization in multi-agent systems. Without modifying agent application source code, MtAgentLens collects process, file, network, and SSL/TLS behaviors in Linux, uses Policy Boundary Crossing (PBC) to quantify sensitive-resource access and inter-agent delegation, and combines a responsibility graph with six-dimensional behavioral scoring to detect attacks and trace causal chains. To systematically evaluate detection under cross-channel payload migration and multi-agent propagation, we design SAFI (Stealthy Attack Framework for Injection), an attack-scenario construction framework covering six attack patterns across three dimensions: review-scope migration, semantic wrapping, and inter-agent propagation. We also construct FuxiBench v2.0, a frozen benchmark containing 58 attack scenarios and 35 benign scenarios. On 29 attacks that reached execution, MtAgentLens achieves a PP-TPR of 0.793 (23/29) and an F1 of 0.852, both higher than all baselines. On 35 benign scenarios, its FPR is 0.057 (2/35, 95% CI [0.016, 0.186]), higher than the 0.029 FPR of R-Judge, the complete application-trace baseline, but lower than those of the three text-based baselines. Ablation results show that the responsibility-graph stage primarily improves ranking (AUROC 0.590→0.677, with the 95% CI spanning zero), whereas the complete decision process primarily improves false-positive control and F1 (PP FPR 0.743→0.057; F1 0.690→0.852). Four representative cases illustrate the process of tracing responsibility chains from alert events back to upstream entry points. Across 16 multi-agent attack scenarios, root-agent localization achieves RootAgent@1, RootAgent@2, and MRR values of 0.688, 0.875, and 0.823, respectively. Under a routine workload with four agents, monitoring adds 3.8% CPU utilization, and mean PSS (Proportional Set Size, a measure of physical memory in actual use) is 115.7 MiB. When monitoring concurrent tasks involving 32 agents, no controlled-operation or persistence losses occur, and task completion remains 100%. These results indicate that MtAgentLens is suitable for Linux multi-agent workflows that produce operating-system-level observable behaviors, complements text- and application-layer defenses, and provides runtime-evidence-based security monitoring and root-cause localization.

Key words: LLM; multi-agent system; eBPF; security monitoring; root cause localization; policy boundary crossing; operating-system-level observability

大语言模型驱动的多智能体系统通过任务委派、上下文交接和子智能体生成等机制组织复杂工作流，已逐步应用于代码评审、根因定位和架构分析与设计语言（Architecture Analysis and Design Language，AADL）模型生成等任务[1]。以 LangGraph[2]为代表的编排框架进一步支持多个智能体（Agent）之间的分工与协作。与此同时，系统的安全边界也由单一提示词文本扩展至智能体间通信、工具执行以及进程、文件和网络资源访问等多个环节。

现有智能体安全研究涵盖提示词检测、安全对齐和应用层运行时防御[3-15]，涉及提示注入、越狱、有害内容和工具调用风险[16,17]。本文关注这些方法的应用层信号之外，如何关联进程、文件、网络与跨智能体交接产生的运行时证据。

为系统刻画并复现攻击载荷（即承载恶意指令、命令参数或敏感操作意图的数据内容）在不同输入通道之间的迁移现象，本文提出载荷迁移分析与攻击场景构造框架 SAFI（Stealthy Attack Framework for Injection）。基于 SAFI，本文构建多智能体安全评测基准 FuxiBench（伏羲基准）v2.0。该基准包含 93 个冻结场景，其中 58 个攻击场景由 SAFI 的 6 种攻击模式及多智能体传播构造生成，用于评估载荷跨通道迁移条件下的运行时监测能力。

按照攻击载荷的主要承载位置，本文区分直接注入（Type A）、文件等外部数据通道中的间接注入（Type

B），以及依赖跨智能体委派、上下文交接或协作关系形成攻击目标的多智能体协同攻击（Type C）。第 2 节给出具体定义；当前 FuxiBench v2.0 评估纳入 Type B 和 Type C。在 Type C 执行集的 9 个攻击中，Agent Sentinel-style[3]检出 1 个（TPR=0.111）。这一结果提示，单条任务文本难以完整呈现跨智能体传播关系。UDora[4]和 ChatInject[5]分别研究动态工具调用观测值和会话模板引入的注入风险；本文进一步关注这些输入经执行和协作产生的进程、文件、网络与上下文依赖关系。

围绕上述风险，现有防御主要沿提示词级检测、安全对齐训练和运行时监测三条技术路径发展。提示词级检测方法通过文本分类识别注入与有害内容，代表性工作包括 LlamaFirewall[6]、Prompt Guard[7]、LLM Guard[8]和 WildGuard[9]；安全对齐方法则利用结构化输入或偏好优化增强模型对注入指令的鲁棒性，如 StruQ[10]、SecAlign[11]和 Meta SecAlign[12]。这些方法主要作用于指令进入执行流程之前。运行时防御方法进一步分析智能体的工具调用、执行事件或交互图，包括 AgentSentinel[3]、MAStitch[13]、BlindGuard[14]和 GUARDIAN[15]，但其观测信息仍主要来自应用层，尚未统一关联进程、文件、网络和 SSL/TLS 行为。相关综述也表明，现有研究已对越狱攻击和对抗性攻击形成较系统的分类[16,17]，但面向载荷跨通道迁移及跨智能体拓扑传播的运行时证据组织仍有待完善。

针对上述目标，本文首先建立统一威胁模型，界定攻击者能力、可信边界、执行轨迹、攻击类型和可观测范围；其中，SAFI 是该威胁模型下面向载荷迁移的场景构造框架。随后，本文提出一种基于扩展伯克利包过滤器（Extended Berkeley Packet Filter，eBPF）[18]的多智能体运行时安全监测与根因定位方法 MtAgentLens。该方法在不修改智能体业务源码的条件下采集 Linux 环境中的进程、文件、网络和 SSL/TLS 行为，并将敏感资源访问与跨智能体委派表示为策略边界穿越（Policy Boundary Crossing，PBC）。在此基础上，MtAgentLens 通过责任图关联智能体身份、资源访问、上下文传播和委派关系，并结合六维行为评分完成安全判定与责任链回溯。

在方法论上，MtAgentLens 借鉴智能运维（Artificial Intelligence for IT Operations，AIOps）中多源事件关联与传播路径回溯的基本思路[19-23]，并面向多智能体安全完成三项扩展：将进程、文件、网络和 SSL/TLS 事件统一映射到智能体身份，以 PBC 量化跨智能体委派触发的敏感行为，并用责任图组织载荷传播、资源访问和责任回溯关系。这些扩展使系统处理的对象由一般故障事件关联转向攻击行为的跨主体、跨资源关联路径，从而同时支持执行期安全监测和告警后的责任链证据查询。

本文围绕四个研究问题开展评估：RQ1，MtAgentLens 对间接注入和多智能体协同攻击的检测能力如何，身份解析与根因智能体定位的效果如何；RQ2，责任图、边界信号和完整判定流程分别对检测产生何种影响；RQ3，文本模式在外部基准 InjecAgent[24]上的表现及其适用边界如何；RQ4，不同并发规模下的监测开销如何变化。

本文的主要贡献如下：（C1）提出 SAFI 载荷迁移分析与攻击场景构造框架，从审查范围迁移、语义包装和跨智能体传播 3 个维度形成 6 种攻击模式；在此基础上构建并冻结 FuxiBench v2.0，其中包含 47 个 Type B、11 个 Type C 攻击场景和 35 个良性场景。（C2）提出并实现 MtAgentLens，在不修改智能体业务源码的条件下，以 eBPF 统一采集进程、文件、网络和 SSL/TLS 事件，并通过多源身份解析、PBC 量化、责任图、污点追踪和六维行为评分实现多智能体运行时安全监测，并支持责任链回溯与根因智能体定位。（C3）从整体检测、分类型检测、组合消融、外部基准、根因智能体定位和性能开销 6 个方面开展评估：在 29 个实际执行攻击上 PP-TPR 为 0.793，在 16 个多智能体攻击场景上根因智能体定位的 RootAgent@1 为 0.688、RootAgent@2 为 0.875，并测量了 1 至 32 个并发智能体规模下的系统开销。

论文其余部分组织如下。第 1 节回顾相关研究；第 2 节给出威胁模型、攻击分类与 SAFI 框架；第 3 节介绍 MtAgentLens 的系统设计；第 4 节说明关键算法；第 5 节和第 6 节分别给出实验设置与评估结果；第 7 节讨论方法的适用边界；第 8 节作为结束语，总结本文工作及后续研究方向。

1 相关工作

与 MtAgentLens 直接相关的研究主要分布在 LLM 智能体安全、eBPF 可观测性、多智能体监测与 AIOps 根因定位三个方向。前两类工作分别提供了攻击面、防御机制和系统级观测基础，后一类工作提供了多源事件关联、异常检测和因果链回溯的方法参照。综合来看，已有研究已经能够覆盖提示注入、越狱攻击、工具调用滥用、应用层运行时防御和系统调用级观测等问题，但面向跨智能体委派、上下文交接和资源访问链的操作系统级安全监测仍需要进一步建模。

1.1 LLM智能体安全：从攻击面到防御谱系

LLM 智能体攻击研究首先从提示注入和越狱攻击展开。AutoDAN[25]、TAP[26]和 GCG[27]分别利用遗传搜索、攻击树搜索和梯度搜索生成高成功率攻击提示词，表明单轮文本输入本身可以构成有效攻击面。随着 LLM 应用从问答系统发展为工具增强型智能体，攻击载体进一步扩展到工具返回内容、文件输入和模板结构。InjecAgent[24]对间接提示注入进行系统评估，揭示外部数据中的恶意指令可以通过工具链影响智能体行为；UDora[4]和 ChatInject[5]则分别讨论动态工具调用劫持和聊天模板滥用。AgentHarm[28]从危害类别角度扩展了智能体安全评测，Formalizing[29]所提出的 LLM-Integrated Application 抽象，则为区分 instruction、untrusted data 和执行响应提供了基础。国内研究中，台建玮等[16]和李南等[17]对 LLM 对抗性攻击与越狱攻击进行了综述，王路桥等[1]则从多智能体协作代码评审的应用场景展示了智能体工程化落地形态。

多智能体场景进一步改变了攻击的组织方式。SoK[30]从整体攻防图景梳理了 LLM 智能体安全问题，Disguise While Defense [31] 将攻防过程放入多智能体博弈，Propagating Unsafe Actions[32]则指出具身智能体中的不安全动作可能在多轮决策中传播。与单智能体攻击相比，多智能体攻击的判定对象不再只是单条提示词或单次工具调用，还包括委派链、角色分工、上下文交接和共识关系。该变化构成本文 Type C 攻击分类的直接问题背景。

防御方法大致形成了三条技术路线。第一类是提示词级检测，以 LlamaFirewall[6]、Prompt Guard[7]、LLM Guard[8]和 WildGuard[9]为代表，其核心是对输入或输出文本进行风险分类。这类方法部署成本低，适合直接注入、越狱文本和有害内容生成检测，但对文件读取后才暴露的载荷、进程级副作用和跨智能体拓扑关系掌握有限。第二类是安全对齐与输入结构化方法，StruQ[10]、SecAlign[11]和 Meta SecAlign[12]通过结构化指令边界或偏好优化增强模型对注入的鲁棒性；Caging the Agents[33]从能力约束出发限制智能体可执行动作，UniGuardian[34]尝试构建统一防护框架。该路线强调执行前约束，适用于降低攻击进入工作流的概率。第三类是运行时防御，AgentSentinel[3]、MAStitch[13]、BlindGuard[14]和 GUARDIAN[15]将监测推进到智能体执行阶段，其中 GUARDIAN 的时序图思路与本文责任图具有相近的问题意识。上述方法为智能体安全提供了重要基础，但多数仍以文本、API 日志或单智能体上下文为主要分析对象。

评测与补充研究进一步丰富了该领域。AgentDojo[35]、EnIGMA[36]、LLM Strategically Deceive[37]、Not What You Signed Up For[38]和 SmoothLLM[39]分别从基准环境、交互式任务、策略性欺骗、用户意图偏离和扰动平滑等角度刻画智能体风险。以上工作表明 LLM 智能体安全已经从单次文本分类问题扩展为覆盖任务环境、工具链和行为后果的系统问题。目前，在将监测对象行为映射到操作系统级行为事件，并进一步把这些事件组织为跨智能体可回溯结构方面仍存在不足。

1.2 eBPF可观测性与安全监测

eBPF 通过 kprobe、tracepoint 和 uprobe 在内核态运行受限程序，能够在较低侵入条件下采集系统调用、进程、网络和用户态函数事件。在云原生安全监测中， Falco[40]等云原生安全工具将 eBPF 用于运行时审计、策略执行和威胁检测。EProbe[41]进一步将 eBPF 用于云原生环境中的容器状态探测，通过内核级事件增强容器状态识别的准确性。AgentSight[42]将 eBPF 引入 LLM 智能体可观测性，通过 uprobe 捕获 SSL/TLS 明文片段，并把 LLM API 调用与系统调用进行关联，证实了 eBPF 进入智能体运行时观测链路的可行性。

与传统主机审计相比，LLM 智能体场景对 eBPF 观测提出了新的组织需求：单个系统调用只表明“发生了什么”，而安全判定还需要知道该事件由哪个智能体发起、是否由上游委派触发、是否与文件载荷或上下文传播相关。因此，MtAgentLens 不只采集 SSL/TLS、进程、文件和网络信号，还将这些信号映射到智能体身份、策略边界和责任图边。

eBPF 运行环境本身也需要受到约束。Moat[43]、Hive[44]和 KrakenGuard[45]分别从隔离、访问控制和内核监控安全角度讨论了 eBPF 基础设施的防护，Cross Container Attacks[46]则指出云环境中的 eBPF 可能引入跨容器攻击面。这些研究表明，基于 eBPF 的智能体安全监测需要清楚界定可信计算基和攻击者能力。本文在威胁模型中将操作系统内核与 eBPF 观测基础设施作为可信计算基（TCB），不讨论内核级攻击者对观测平面的直接破坏。

1.3 多智能体系统监测与AIOps方法论迁移

多智能体系统监测研究关注多主体协作中的故障、异常和责任归属。AgenTracer[47]通过反事实重放和故障注入支持多智能体系统的故障归属，AgentTuning[48]从监督微调角度说明智能体能力可以被定向塑造，Kazari 等[49]在合作式多智能体强化学习场景下讨论去中心化异常检测，Electrode[50]则展示了 eBPF 对分布式协议观测与加速的系统价值。这些工作均关注多主体系统中的行为关联；与一般故障归属不同，本文的研究重点聚焦于 LLM 智能体安全事件中的攻击载荷、敏感资源访问和跨智能体传播关系。

AIOps 和根因定位研究为本文提供了方法论参照。AmazeMap[19]以多层次影响图组织系统组件关系，于庆洋等[20]利用调用链控制流进行异常检测；PASER[21]通过多维 KPI 分解分析异常来源，SOTIMiner[22]从时序不变式中发现异常模式，BARO[23]使用多元贝叶斯变点检测增强异常识别的鲁棒性。RCA Causal[51]与 Chain-of-Event[52]则从因果结构和事件链角度支撑根因回溯。国内相关工作中，李晓卓[53]、包航宇[54]、殷康璘[55]和张明悦[56]分别从软件工程、AIOps、系统韧性和多智能体博弈等方向提供了补充背景。

上述研究表明，多源观测、图结构建模和因果链回溯已经在智能运维和分布式系统分析中形成成熟思路。将其迁移到 LLM 多智能体安全场景时，关键差异在于：监测对象由服务调用和 KPI 异常转为提示词、文件、进程、网络和智能体委派关系；分析目标由故障定位转为攻击检测与责任归因；证据结构由服务拓扑转为智能体—操作系统交互图。表 1 概括了相关技术路线的适用边界与本文定位。

表 1 相关工作技术路线、适用边界与本文定位

技术路线	代表工作	主要信号	适用边界	与本文关系
提示词级检测	LLM Guard[8]、WildGuard[9]	输入/输出文本	直接注入、越狱文本、有害内容生成	提供执行前文本侧基线
安全对齐与结构化输入	StruQ[10]、SecAlign[11]、Meta SecAlign[12]	指令边界、偏好数据	降低注入进入模型决策的概率	与操作系统级运行时监测互补
智能体运行时防御	AgentSentinel[3]、GUARDIAN[15]	智能体交互日志、时序图	单智能体或应用层运行时审计	与责任图思想接近，但观测层级不同
eBPF 可观测性	AgentSight[42]、Electrode[50]	SSL/TLS、系统调用、协议事件	运行时低侵入观测	提供操作系统级信号采集基础
AIOps 根因定位	AmazeMap[19]、PASER[21]、SOTIMiner[22]、BARO[23]、Chain-of-Event[52]	多维指标、影响图、事件链	分布式系统异常检测与溯源	支撑多源融合和因果链回溯

总体而言，现有研究已覆盖文本级检测、安全对齐、应用层运行时审计和系统可观测性，但仍缺少一种能够在不依赖特定智能体框架内部状态的条件下，联合关联跨智能体委派、进程派生、文件访问、网络通信与上下文传播证据的方法。如何将这些异构事件组织为可用于在线判定和责任回溯的统一行为链，仍是多智能体运行时安全监测需要解决的问题。

2 威胁模型

本节依次界定攻击者能力、可信边界、执行轨迹和攻击类型，并介绍 SAFI 载荷迁移框架及检测范围，为后续行为建模与实验评估提供统一定义。

2.1 攻击者能力与边界

本文将操作系统内核、eBPF 观测基础设施及其采集配置视为可信计算基，不考虑能够修改内核、直接篡改监测数据或控制监测基础设施的攻击者。在该边界内，攻击者可通过三类用户态入口影响智能体任务：（1）输入控制，即提交提示词文本或任务可访问的外部文件；（2）协作控制，即控制协作图中的一个或多个非可信智能体，使其发送恶意委派、篡改交接上下文或协同确认有害操作；（3）外部交互控制，即提供攻击者控制的命令与控制端点，并诱导具有相应权限的智能体读取、写入或传输数据。上述能力不要求 root 权限，且不超出被测智能体进程原有的文件和网络访问权限。攻击者可在任务允许访问的用户态目录中预置外部文件，或通过任务输入诱导智能体生成文件，但不因此获得系统任意路径写权限。

2.2 执行轨迹与监测问题定义

定义 1（多智能体执行轨迹）给定一次多智能体任务执行，其轨迹记为 $\tau=(A, R, E_{os}, E_{msg}, T)$。其中，$A=\{a_1, ..., a_m\}$为参与执行的智能体集合，R 为文件、网络端点、进程和工具调用等资源集合；E_{os} 为操作系统级可观测事件集合，每个事件可表示为 $e=(p, op, r, t, ev)$，分别对应进程、操作、资源、时间戳和证据属性；E_{msg} 为智能体间消息、委派和上下文交接事件集合；T 为事件偏序关系，用于刻画执行中的先后约束。本文的操作系统级监测主要依赖 E_{os}，并通过 SSL/TLS 明文片段、进程树和工具调用参数推断 E_{msg} 中可观测的部分。

定义 2（操作系统级可观测攻击）设 B 为策略边界集合，包含敏感文件、外部网络端点、凭据访问、跨智能体委派和上下文传播规则。若一次攻击在执行轨迹 τ 中产生至少一个与攻击目标相关、可由监测器记录的运行时行为，例如进程创建或执行、文件读写、网络连接、TLS 数据交换或工具调用，则称该攻击具有操作系统级可观测行为。仅改变模型输出文本且未触发上述行为的攻击，不属于 MtAgentLens 的独立检测范围。被模型安全过滤器在执行前阻断且未产生操作系统级事件的攻击，也不属于该范围。

基于上述定义，本文研究的问题可表述为两个函数：安全判定函数 $f(\tau, B)\rightarrow y$，其中 $y\in\{$ Safe, Unsafe, Uncertain $\}$，分别表示安全、不安全和待复核状态；溯源函数 $g(\tau, B)\rightarrow\langle a_0,e_1,a_1,\ldots,e_k,r_k\rangle$输出一条按图中的依赖关系与时间约束组织的责任链。其中，a_0 表示能够追溯到的最早载荷入口或触发智能体，即上游智能体；e_i 表示改变载荷传播、智能体委派、进程派生、敏感文件访问、外部网络连接或凭据使用的关键事件，且至少由一条带时间戳的运行时记录支持；a_i 表示在委派、上下文交接或载荷传播关系中接收上游输入并继续转交或执行相关动作的下游智能体；r_k 表示由策略边界集合 B 给出的最终受保护资源。本文以带时间戳的进程、文件、网络和 SSL/TLS 事件作为直接运行时证据，再基于进程层级、会话标识和消息关联推断智能体间交互，并保留原始事件和置信度。MtAgentLens 不对未触发进程、文件、网络或 SSL/TLS 行为的纯文本有害内容作

独立判定；提示词级检测器负责执行前文本审查，MtAgentLens 负责执行期行为监测与责任链回溯，两者覆盖不同阶段。

2.3 三类操作型攻击：按载荷载体分类

定义 3（载荷载体与攻击类型）基于 Formalizing[29]提出的 LLM-Integrated Application 模型（instruction prompt+untrusted data→LLM→response），将 LLM 智能体攻击按载荷首次进入执行轨迹的载体分为互斥的三类（表 2）。若载荷进入用户提示词文本，则归为 Type A；若进入随任务提交或工具读取的任务辅助文件（fixture files，指随评测任务提供、供智能体读取或处理的预置文件），则归为 Type B；若恶意性主要由智能体间委派、上下文交接、共识操纵或子智能体生成关系形成，而单条提示词或单个文件不足以完整表达攻击目标，则归为 Type C。三类攻击以首要载体和判定所需上下文为分类依据；当场景同时包含多个载体时，以形成攻击语义所必需的最高阶载体为准。

表 2 三类操作型攻击按载荷载体分类

类型	攻击向量	载荷载体	提示词可见性	文本级检测可行性
Type A—直接注入	攻击者提示词文本	用户提示词	可见	可由文本级方法处理，但存在漏报
Type B—间接注入	随附文件	任务辅助文件	文件读取前不可见	需要文件或工具读取证据
Type C —多智能体协同	智能体间交互拓扑	委派/上下文/共识关系	单条提示词不充分	需要跨智能体拓扑证据

Type A（直接提示注入）以 AgentHarm[28]的攻击任务为代表，关注恶意指令直接出现在用户提示词文本中的攻击。此类攻击不依赖文件或工具输出，重点考察操作系统级信号对提示词文本攻击的补充检测能力。

Type B（间接注入/SAFI）关注载荷从提示词通道迁移到任务辅助文件后的检测问题。此时提示词文本描述合法开发任务，但文件内容携带恶意载荷。InjecAgent[24]、UDora[4]和 ChatInject[5]从不同角度展示了间接注入、工具调用劫持和模板结构滥用等攻击向量。

Type C（多智能体协同）的攻击语义依赖智能体间的委派、上下文交接、共识操纵或子智能体生成关系。攻击可表现为共谋环、角色劫持、多级委派和碎片化载荷传播等形式。具有多智能体执行拓扑的文件注入仍可归入 Type B；只有跨智能体关系对形成攻击语义不可缺少时才归入 Type C。当前基准包含 11 个 Type C 攻击场景。

2.4 SAFI框架：载荷迁移与安全不对称性

SAFI（Stealthy Attack Framework for Injection）在本文中作为基准构造与载荷迁移分析框架，用于刻画 LLM 智能体工作流中不同数据通道的审查强度差异。表 3 概括本文预实验观察到的四级审查边界不对称性。

表 3 SAFI 框架：四级审查边界不对称性

数据通道	审查强度	典型观察	审查主体
用户提示词文本	深度扫描	拒绝较多	LLM 安全过滤器
工具调用参数	部分扫描	拒绝不稳定	框架输入验证
文件内容（工具读取）	较少显式扫描	通常未见明确拒绝	通常未显式配置
Bash 命令输出	较少显式扫描	通常未见明确拒绝	通常未显式配置

SAFI 使用三类载荷迁移策略刻画上述不对称性：（1）审查范围迁移，将载荷从提示词通道移至文件或输出通道；（2）语义包装，通过语言转换或语义拆分（P6 载荷拆分）降低单步文本审查的可见性；（3）跨智能体传播，利用多智能体工作流的时间展开将攻击影响分散到多个合法步骤中。

SAFI 包含 6 种具体攻击模式（P1～P6），覆盖文件注入、配置伪装、构建验证、代码生成、语言包装和载荷拆分等形式。其作用在于将 Formalizing[29]的两通道模型（instruction+data）扩展为更细粒度的审查边界模型，说明文本级防御在 Type B/C 攻击上需要额外的文件、工具调用或跨智能体上下文证据。

2.5 检测范围

MtAgentLens 的独立检测范围是产生进程、文件、网络或 SSL/TLS 行为的攻击。评估使用场景钩子标记（marker）作为执行代理：58 个攻击场景中 29 个出现执行标记，构成执行集；其余 29 个仍纳入全场景结果统计。标记用于确认预定执行路径是否到达，不单独证明攻击目标已经实现。

3 系统设计

本节围绕多智能体应用语义与操作系统级事件之间的映射问题，介绍 MtAgentLens 的设计目标、总体架构和分层处理流程。系统按照“事件采集—证据组织—风险判定”划分为 L1、L2 和 L3 三层：L1 以低侵入方式采集操作系统级行为并提取 PBC 证据，L2 完成智能体身份解析、责任图构建与责任链追踪，L3 融合多源行为信号并适时进行校准。三层设计分别落实零插桩、框架弱依赖、行为证据优先和校准解耦等设计目标。

3.1 设计动机：构建智能体行为—操作系统级事件映射

多智能体框架以任务、委派和上下文描述应用层行为，操作系统则分别记录进程、文件、网络和 SSL/TLS 事件。MtAgentLens 需要在不修改业务源码的条件下，将分散的底层事件映射回具体智能体、任务与委派链，从而保留行为语义并支持跨层事件关联。具体而言，LLM 推理通常伴随 SSL/TLS 通信，工具调用会产生进程创建或命令执行，文件读写会触发 VFS 操作，敏感文件或外部网络资源访问则可能形成策略边界穿越（PBC）。该映射使操作系统层成为跨智能体工作流的公共监测平面。

为构建智能体行为与操作系统级事件之间的映射，系统综合利用进程标识与会话信号完成智能体身份关联。其中，pid 用于标识承载智能体行为的进程实例，tls_session 用于关联 SSL/TLS 明文及其对应的 API 会话，conversation_id 用于标识一次任务对话或上下文交接链。系统结合进程创建关系，对 pid、tls_session 和 conversation_id 之间的关联进行并查集融合，生成稳定的 agent_id。该标识用于 PBC 事件归属、责任图构建与污点传播。

3.2 设计原则与总体架构

MtAgentLens 遵循四项设计原则：（1）业务源码零侵入，无需修改智能体业务源码或增加应用层插桩，监测主要通过 eBPF 探针实现；（2）框架弱依赖，采集层不绑定 LangGraph[2]等特定多智能体编排框架，而以进程、文件、网络和 SSL/TLS 等操作系统级信号作为统一入口；（3）行为证据优先，检测主要依据运行时行为信号，文本语义分析仅作为辅助校准手段；（4）校准解耦，检测决策与语义评估分层，降低 LLM 仲裁不确定性对基础信号通道的影响。

总体架构为三层流水线（图 1）。

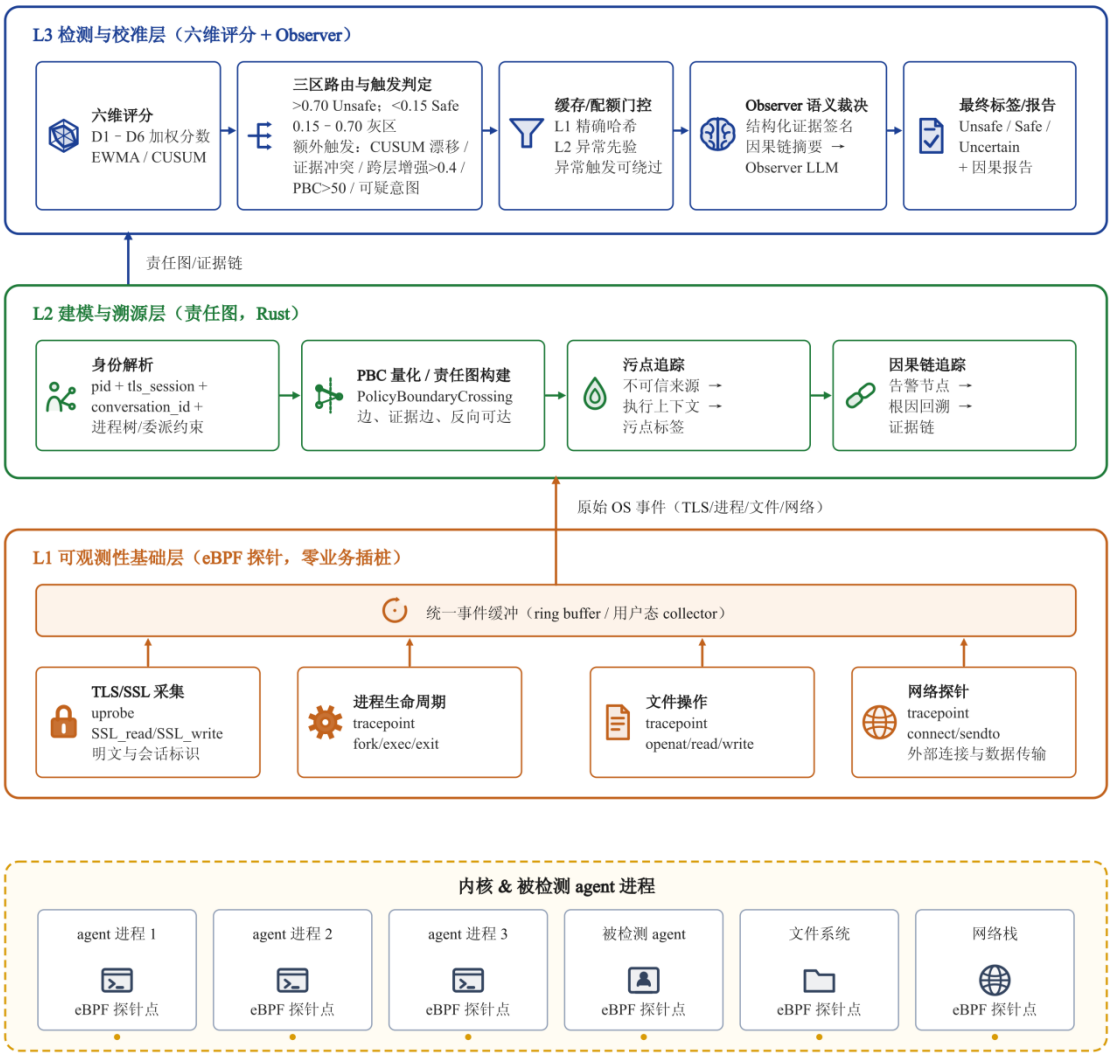


图 1　MtAgentLens 三层流水线架构

L1（可观测性基础层）通过 SSL/TLS、进程、文件和网络探针采集操作系统级事件；L2（建模与溯源层）完成身份解析、PBC 量化、责任图构建、污点追踪和责任链追踪；L3（检测与校准层）通过六维行为评分体系给出安全判定，并仅在灰色地带（即安全阈值与危险阈值之间的不确定分数区间）、跨层证据增强、累积和 CUSUM 漂移或证据冲突等预设条件下触发 Observer 语义仲裁。三层之间形成从事件采集、证据组织到风险判定的递进关系。

3.3 L1：操作系统级信号采集

3.3.1 eBPF 探针体系

L1 按“信号—挂载点—字段—用途”组织为四类：SSL/TLS 探针通过 uprobe 挂载 SSL_read 和 SSL_write，提取会话标识与明文片段，用于建立 tls_session；进程探针通过 tracepoint 捕获进程创建、执行和退出事件，

记录 pid、ppid、comm 和 start_time，用于身份解析与委派重建；文件探针监测 openat、read 和 write 等操作，用于识别敏感文件访问；网络探针监测 connect 和 sendto 等操作，用于记录外部连接和疑似数据传输。探针采集的事件统一进入 L2 证据组织层。

3.3.2 PBC 量化机制（跨 L1/L2）

PBC（Policy Boundary Crossing）用于把智能体行为中的策略边界穿越转化为可统计信号。

定义 4（策略边界穿越）给定智能体 a、动作 op、资源 r、时间 t 和策略边界集合 B，定义指示函数 PBC(a, op, r, t; B)。当动作(a, op, r, t)越过策略边界 B 时，PBC(a, op, r, t; B)=1；否则 PBC(a, op, r, t; B)=0。这里“越过策略边界 B”表示动作访问了预定义敏感文件、外部网络端点或凭据资源，越过文件/网络访问策略，或者由跨智能体委派触发了敏感执行行为。每一次 PBC=1 的事件在责任图中对应一条 Agent→PolicyBoundary 的 PolicyBoundaryCrossing 边，目标节点记录边界类别（如文件读、文件写、网络连接），检测统计则采用统一的 PBC 计数口径。

PBC 覆盖三类与威胁模型直接相关的行为路径：多智能体委派会引入编排工具调用、进程创建和上下文交接；文件载荷注入要求智能体读取或处理任务辅助文件；命令与控制、数据泄露和敏感凭据访问通常表现为网络连接或敏感路径访问。良性任务也可能产生边界访问，因此 PBC 作为六维评分中的行为证据参与判定。

在单次任务会话内，责任图只追加经验证的 PBC 边且不删除历史边，因此 $N_PBC(t+1) \geq N_PBC(t)$。身份合并只改变边的主体归属，不减少会话内已记录边数。该性质使 PBC 可以同时服务于执行期告警和责任链回溯；其检测贡献在第 6.2 节验证。

3.4 L2：责任图与身份解析

3.4.1 责任图数据结构

责任图（ResponsibilityGraph）是一种面向多智能体系统的归因溯源有向异构图。责任图用于把分散在不同进程和数据通道中的事件组织成可反向查询的攻击链。难点在于操作系统级事件不直接携带智能体委派语义，且意图、执行、传播和资源访问分属不同事件；为保留“谁触发—如何传播—触达何种资源”，图中分别建模主体、执行载体、上下文、资源及其关联关系。除通用实体名称外，下述类型集合及组合方式均为本文面向多智能体溯源定义。

定义 5（责任图）给定执行轨迹 τ，责任图表示为 $G=(V, E, \phi, \psi, \rho)$。其中，$V$ 为实体节点集合，$E \subseteq V \times V$ 为有向边集合，$\phi: V \rightarrow T_V$ 为节点类型映射，$\psi: E \rightarrow T_E$ 为边类型映射，ρ 为边到证据、时间戳和置信度的属性映射。责任图的整体结构、节点类型及边类型如图 2 所示。

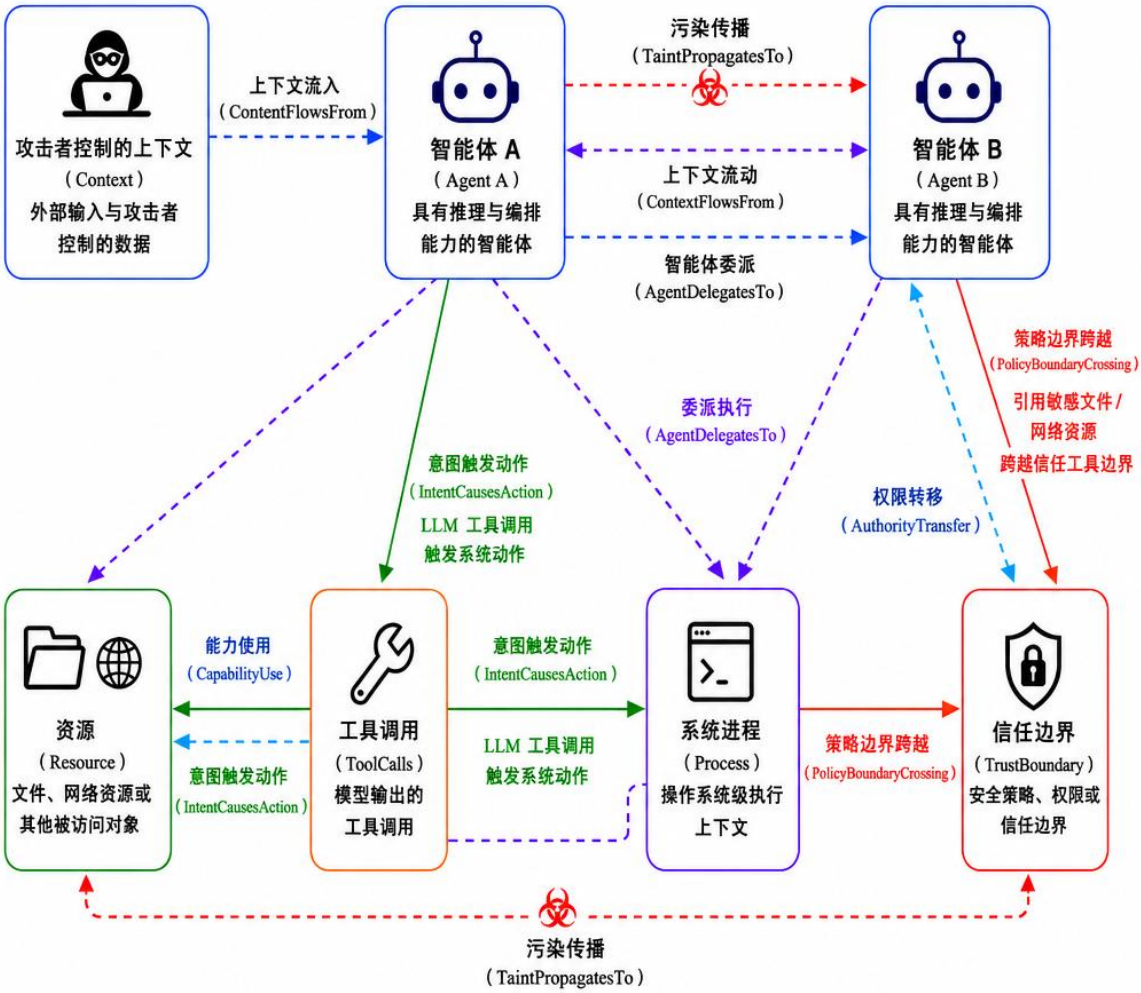


图 2　责任图结构示意

围绕安全监测任务，责任图重点组织三类证据边：（1）IntentCausesAction，将 LLM 返回的 tool call 与操作系统级动作配对，连接“意图”和“行为”；（2）PolicyBoundaryCrossing，刻画敏感文件、网络资源或跨智能体委派触发的策略边界穿越；（3）TaintPropagatesTo，追踪攻击者控制的数据被读取后在智能体间传播的路径。AgentDelegatesTo、ContextFlowsFrom、CapabilityUse、AuthorityTransfer 等边则用于表达委派关系、上下文继承和能力使用。

在给定证据映射 ρ 的条件下，责任图把离散操作系统级事件组织为可回溯结构：检测事件可以沿 IntentCausesAction 与 PolicyBoundaryCrossing 回溯至责任智能体，也可以沿 TaintPropagatesTo 和 ContextFlowsFrom 定位受影响的后续智能体。责任链查询可表示为从风险资源节点或 PolicyBoundary 节点出发，在 G 上寻找满足边类型约束的反向可达路径。

3.4.2 身份解析与责任链追踪

eBPF 记录的原始主体是进程或 SSL/TLS 会话，而安全监测需要回答“哪个智能体在何种上下文中触发了该行为”。因此，原始进程、SSL/TLS 会话与任务会话之间并非一一对应：同一智能体可能派生多个工具进程并建立多个 SSL/TLS 会话，不同智能体也可能在同一宿主进程中交替执行，从而导致仅凭单个 pid 或单个 tls_session 无法准确确定智能体身份。为此，系统将 pid、tls_session 与 conversation_id 作为身份解析的基础关联字段，并结合进程树关系、会话归属关系和上下文交接关系，将底层操作系统事件映射至相应的智能体级主体，为跨进程、跨会话和跨智能体的证据关联建立统一索引。

身份解析采用基于并查集的多源融合方法。系统先按 SSL/TLS 会话建立进程身份分量，再沿进程树传播身份；无独立会话的工具子进程继承父智能体身份。对于具有独立 TLS 会话且满足智能体进程识别条件的子进程，系统保留独立 agent_id，并记录父子智能体边界。conversation_id 及委派事件用于关联上下文。仅凭进程分叉或单个会话标识不足以识别共享进程、连接复用下的全部逻辑智能体；证据不足的单元保留为 unknown。

污染追踪器（TaintTracker）标记受攻击者控制的数据，并追踪其在文件读取、上下文传递和跨智能体调用中的传播路径。责任图查询以 PolicyBoundaryCrossing 边定位风险事件，再沿 IntentCausesAction、AgentDelegatesTo 和 ContextFlowsFrom 等边回溯恢复事件链，从而将单点异常告警整合为可追溯的攻击路径。

3.5 L3：多信号融合监测引擎

MtAgentLens 的 L3 层采用六维行为评分体系进行安全与否的评估与判定，将 L1 的操作系统级信号和 L2 的图模型信息融合为统一检测分数（表 4）。

表 4 六维行为评分体系

维度	内容	权重	设计依据
D1—边界严重性	异常边占比与不安全规则命中率取较大值	0.28	与攻击目标直接相关的策略边界穿越
D2—内容威胁	注入/内容模式匹配置信度均值	0.25	文本/应用层注入模式识别
D3—智能体放大	PBC/意图数的 sigmoid×证据质量	0.20	攻击将 PBC 集中于少数工具调用
D4—观测累积	min(N_obs/50,1)	0.10	任务内证据累积规模代理，不表示重复越界次数
D5—跨上下文佐证	匹配级别加权的证据置信度	0.10	L1=1.0/L2=0.85/L3=0.7/L4=0.15/L5=0.02
D6—行为基线偏离	证据支持的 PBC 与高置信关联记录的归一化聚合	0.07	凭据扫描/数据外泄等行为模式

六个维度分别描述策略边界、内容模式、智能体放大、观测累积、跨上下文佐证和行为证据。正常开发任务也会访问文件与网络，多维融合用于结合行为强度与证据质量进行判定。

评估采用归一化权重向量 w=(0.28,0.25,0.20,0.10,0.10,0.07)。D1 至 D3 侧重策略边界、载荷内容和执行放大，D4 至 D6 补充观测规模与证据支持。权重在冻结评估前确定，敏感性分析见第 6.2 节。

本次冻结评估将六维子分数 d_j 按权重 w_j 作线性加权，并在 d_j>0.01 的活跃维度上计算加权几何平均；两者各占 0.5，得到原始分数 s_raw。随后以 s_cal= σ (7s_raw−3)作 sigmoid 映射，其中 σ (x)=1/(1+exp(−x))。最小证据门控为 g= σ ((N_PBC−10)/2)，存在 Unsafe 规则命中时取 g=max(g,0.3)，门控分数为 s_gate=min(s_cal • g,1)。

D4 使用传入评分器的观测计数 N_obs，并按 50 归一化截断至[0,1]。该计数刻画任务内的观测累积，区别于 PBC 数量。

渐进评分器以指数加权移动平均（EWMA）和累积和（CUSUM）维护智能体的时序信号，用于触发额外审查；分类评分器计算六维基础分数，用于三区路由。分数描述证据强度，最终标签由路由和审查结果共同确定。

基础分数低于安全阈值 0.15 时进入 Safe 路由，高于危险阈值 0.70 时进入 Unsafe 路由，其余情况由 Observer 仲裁。Observer 返回 Unsafe 或 Safe 时采用相应标签；返回 Uncertain 时保留待复核状态，调用失败时回退至基础分数的三区规则并记录失败。最终确定的强规则违规保留其 Unsafe 判定。评估读取最终判定字段，以 Unsafe 计为二元告警，不以仲裁后的数值是否高于 0.15 重新定标签。

除灰色区外，CUSUM 漂移、证据冲突、跨层增强（progressive>0.4）、PBC>50 或可疑意图也可触发 Observer。系统使用结构化签名去重和资源模式缓存复用已有判定，并在异常触发时绕过缓存。Observer 输入包括图与规则摘要、关联证据和可用的资源或工具意图信息。

4 关键算法

围绕第 3 节的三层架构，本文将核心过程归纳为三个算法：算法 1 完成多源身份解析，算法 2 完成 PBC 抽取与责任图更新，算法 3 完成六维融合判定与 Observer 校准。

算法 1 多源信号驱动的 Agent 身份解析

输入：冻结信号快照 $S = (P, E_{\text{proc}}, E_{\text{tls}}, E_{\text{msg}})$，阶段 1 临时身份映射 M_p
输出：权威身份集合 A，进程到 Agent 的映射 M_a，子 Agent 边界集合 B_s

行号	步骤
1	初始化身份分量集合 C、进程身份映射 M_a 与子 Agent 边界集合 B_s。
2	提取 $P_{\text{tls}} = \{p \mid p \in P \wedge TLS(p) \neq \varnothing\}$，并将其中进程初始化为并查集分量。
3	对每个 TLS 会话 t，令 $P_t = \{p \mid t \in TLS(p)\}$，并将 P_t 中共享同一会话的进程合并到同一身份分量。
4	对每个种子分量 $c \in C$ 执行进程树向下传播；若子进程 q 无 TLS 会话，则继承父进程身份；若 $TLS(q) \cap TLS(p) \neq \varnothing$，则合并 p 与 q。
5	若子进程 q 具有独立 TLS 会话且进程名属于 Agent 进程集合，则保留 q 的独立身份，并将父子 Agent 边界加入 B_s；否则将 q 作为工具进程归入父 Agent。
6	对仍未归属的进程，沿父进程链向上查找至多 5 层；若找到已归属且非共享基础设施的祖先进程，则继承其身份，否则记录为未解析进程。
7	对每个非空身份分量，按 $(pid, start_time)$ 排序并计算 SHA256 前缀，生成稳定 Agent 标识；同时收集对应 TLS 会话、消息标识和阶段 1 临时身份引用。
8	返回 A、M_a 和 B_s。

复杂度：并查集合并的摊还复杂度为 $O(\alpha(n))$，其中 α 为反 Ackermann 函数；进程树传播与身份导出的复杂度为 $O(|P|+|E_{\text{proc}}|)$。

算法 1 先将 SSL/TLS 会话、进程层级和消息标识统一到稳定的智能体身份上，并显式保留子智能体边界。该身份映射为后续 PBC 归属、责任图建边和跨智能体污点传播提供统一索引。

算法 2 PBC **抽取与责任图增量更新**

输入：相关证据 e，意图 i，动作 a，身份映射 M_a，当前责任图 G，策略集合 $\mathcal{B}$
输出：更新后的责任图 G'，图增量 Δ，PBC 边集合 E_{pbc}

行号	步骤
1	初始化图增量 Δ 与 PBC 边集合 E_{pbc}。
2	由 M_a 解析意图进程和动作进程对应的 Agent，记为 $agent_i$ 与 $agent_a$。
3	在 G 中创建或更新 $agent_i$、$agent_a$ 对应的 Agent 节点。
4	将动作类型与规范化参数转换为资源 r，并创建或更新 Resource 节点。
5	插入 IntentCausesAction 边，连接意图 Agent 与动作资源，并绑定证据 e。
6	计算 $state \leftarrow evaluate(r, \mathcal{B})$。
7	若 $state \in \{\text{Observed}, \text{Ambiguous}\}$，且资源不属于工作区文件或系统解析副作用，则创建或更新 PolicyBoundary 节点；若同一 Agent-边界对已有临时边，则用证据 e 更新该边，否则插入 PolicyBoundaryCrossing 边并加入 E_{pbc}。
8	若意图内容触发内容边界，则创建对应 PolicyBoundary 节点并更新 PolicyBoundaryCrossing 边；若动作为文件写入或覆盖，则将目标文件标记为污染源。
9	若意图调用编排工具且 $agent_i \neq agent_a$，则执行委派边界判定，并在匹配时插入 PolicyBoundaryCrossing 边。
10	若动作是跨 Agent 文件读取且目标文件已标记为污染源，则插入 TaintPropagatesTo 边并更新上下文信任状态。
11	更新责任图索引和 Δ，并返回 G'、Δ 与 E_{pbc}。

复杂度：单条证据的节点查找、边插入与索引维护平均为 $O(1)$；污点检查和边界判断与资源索引规模相关。

算法 2 生成的 PBC 与责任图统计进入渐进评分器。时序状态采用 EWMA（α =0.1）和 CUSUM 维护，事件缓冲保留最近 200 条记录，时间窗口为 600 s，指数衰减半衰期为 300 s。算法 3 给出六维融合、sigmoid 映射、最小证据门控和三区路由。

算法 3 六维融合判定与 Observer **校准**

输入：规则命中集合 R，图模式集合 P，责任图 G，证据集合 E，观测计数 N_{obs} 及意图信息
输出：判定结果 $y \in \{Safe, Unsafe, Uncertain\}$，基础分数 s_{gate}，路由标识 $route$

行号	步骤
1	按表 4 计算六维子分数 $d = (d_1, \ldots, d_6)$，取 $w = (0.28, 0.25, 0.20, 0.10, 0.10, 0.07)$。
2	计算线性分数 $s_{lin} = \sum_j w_j d_j / \sum_j w_j$。
3	令 $J = \{j : d_j > 0.01\}$；在 J 上计算加权几何平均 s_{geo}，若 $J = \varnothing$ 则取 0。
4	计算 $s_{raw} = 0.5 s_{lin} + 0.5 s_{geo}$，并映射 $s_{cal} = 1/[1 + \exp(-(7 s_{raw} - 3))]$。
5	计算门控 $g = 1/[1 + \exp(-(N_{PBC} - 10)/2)]$；若存在 Unsafe 规则命中，则令 $g = \max(g, 0.3)$。
6	计算基础分数 $s_{gate} = \min(s_{cal} \cdot g, 1)$。
7	若存在已确认的强规则违规，保留其 Unsafe 判定及证据引用。
8	否则，若 $s_{gate} < 0.15$ 则进入 Safe 路由；若 $s_{gate} > 0.70$ 则进入 Unsafe 路由。
9	若 $0.15 \leq s_{gate} \leq 0.70$ 或满足附加触发条件，则构造结构化证据摘要并调用 Observer。
10	Observer 返回 Unsafe、Safe 或 Uncertain 时，分别保留对应判定及仲裁状态。
11	调用失败时记录失败，并按基础分数回退：> 0.70 为 Unsafe，> 0.15 为 Uncertain，其余为 Safe。
12	返回最终判定；二元评估仅将 Unsafe 计为告警，不以仲裁后分数重新定标签。

说明：本算法采用本次冻结评估的融合与门控口径；额外触发和缓存机制见第 3.5 节。

Observer 缓存采用两级策略。L1 哈希去重将结构化签名映射到已产生的判定结果，相同签名可直接复用，并在任务边界处清空；L2 按资源模式累积 Safe/Unsafe/Uncertain 判定分布，计算非 Safe 判定占比，并通过 CachePrior（缓存先验模块）向 Observer 提供历史异常率先验。

MtAgentLens 原型在 Linux 上实现。eBPF 探针使用 libbpf CO-RE（Compile Once, Run Everywhere）框架编写[18]；流式事件处理与责任图引擎使用 Rust 实现，并基于 tokio 异步运行时处理事件流。

5 实验设置

本节介绍 FuxiBench v2.0、基线及其输入、运行环境和评估协议。

5.1 FuxiBench v2.0基准

FuxiBench v2.0 是本文构建的冻结基准数据集，用于评估多智能体安全监测与根因定位能力。基准包含 93 个场景，包括 58 个攻击与 35 个良性场景，30 个场景采用多智能体执行。所有阈值、评分参数和基线输入清单均在冻结评估前确定；基准冻结后不再根据该批测试结果调整这些参数。

FuxiBench v2.0 包含 47 个 Type B 和 11 个 Type C 攻击场景，以及 35 个良性场景(FuxiBench v1.0 中 Type A 攻击场景目前已无法通过大部分大模型本身安全校验，因此本次数据集升级移除了 Type A 攻击场景)。Type B 为经文件、工具返回或任务辅助文件迁移的间接注入；Type C 为借助跨智能体委派、上下文交接与协作关系形成的多智能体协同攻击。35 个良性场景包括 26 个原生良性任务和 9 个经重新审查归为良性的历史攻击设计，覆盖正常多步任务、授权敏感访问等边界情形。

表 5 汇总了 FuxiBench v2.0 的场景构成、载荷或交互载体及其评估重点。

表 5 FuxiBench v2.0 场景构成

场景类型	数量	载荷或交互载体	评估重点
Type B 间接注入	47	SAFI 任务辅助文件	载荷迁移到文件通道后的检测能力
Type C 多智能体协同	11	智能体间委派、上下文交接与共识过程	跨智能体拓扑攻击的监测与溯源能力
良性场景	35	正常多步开发任务	误报控制能力
合计	93	58 攻击+35 良性	检测、误报与根因定位综合评估

每个场景包含四类材料：（1）智能体配置，包括 1～5 个智能体、LangGraph[2]等框架配置以及 DeepSeek v3 等模型设置；（2）任务描述，即与攻击载荷分离的合法开发任务；（3）攻击注入方式，对应 Type A/B/C 的不同载体；（4）真值（ground truth）标注。标注由 3 位安全研究人员按 GT v3 协议独立完成，分歧通过多数投票解决，Fleiss' κ =0.87。对于 Type B 和 Type C，标注同时参考场景设计目标和独立运行日志，以确认攻击是否到达执行层；标注过程不使用 MtAgentLens 的风险分数、PBC 计数或 Observer 判定作为标签来源。

5.2 基线系统

本文比较四个基线：（1）Agent Sentinel-style[3]，复现 AgentSentinel 的任务文本审计提示与结构化输出流程；（2）LLM Guard[8]，使用本地 PromptInjection v2 权重与相应窗口和阈值逻辑；（3）Meta Prompt Guard，使用本地 Prompt-Guard-86M 权重[7]；（4）R-Judge[57]，按两步审计协议接收任务、模型输出、工具调用及返回。LLM 审计基线使用记录中的 DeepSeek Flash 配置，Agent Sentinel-style 的结果不代表 AgentSentinel 完整沙箱系统。

Agent Sentinel-style 接收 task 文本，LLM Guard 和 Meta Prompt Guard 接收 task 与 fixture 文本；R-Judge 接收整次运行的应用轨迹，MtAgentLens 接收操作系统事件及关联意图。各方法的观测范围不同，比较结果同时反映输入信息、模型与判定流程。

5.3 评估协议与指标

检测评价使用执行集真阳性率（PP-TPR）、误报率（FPR）、精确率与召回率的调和平均 F1，以及受试者工作特征曲线下面积（AUROC）。执行集由 29 个出现执行标记的攻击与 35 个良性场景组成，PP-TPR、精确率和 F1 在该集合上计算；FPR 以 35 个良性场景为分母。Uncertain 按非 Unsafe 处理并另报复核数量。AUROC 用于连续分数排序；二元率的 95%置信区间采用 Wilson 法。

根因智能体定位采用 RootAgent@1、RootAgent@2 和 MRR 三项指标：RootAgent@k 表示真实根因集合中至少一个智能体是否位于候选排序前 k 位；MRR 为每个场景首次命中任一参考根因的排名倒数的均值，未命中记为 0。三项指标在具有攻击效应且存在多个候选智能体的多智能体攻击场景上计算。

整体二元判定在执行集的 64 个场景上使用配对精确 McNemar 检验，四项基线比较采用 Holm 校正。消融 AUROC 的相邻层比较使用配对 DeLong 检验及 Holm 校正，区间按场景 bootstrap 计算。安全与危险阈值分别为 0.15 和 0.70。实验平台为 Intel i7-13700K、64 GB RAM 的 Linux 环境。

6 实验评估

本节依次报告安全检测与根因主体定位、组件贡献、外部基准表现及监测开销。

6.1 RQ1：安全监测与根因定位能力

RQ1 在 FuxiBench v2.0 上评估整体与分类型检测结果，再分别考察身份解析和根因主体定位。

6.1.1 整体检测能力

图 3 和表 6 给出整体检测结果。MtAgentLens 执行集 PP-TPR 为 0.793（23/29），FPR 为 0.057（2/35），F1 为 0.852、精确率为 0.920；R-Judge 的 TPR/F1 为 0.552/0.696，Agent Sentinel-style 与 LLM Guard 的 PP-TPR 均为 0.138，Meta Prompt Guard 为 0.517。执行集上的二元判定比较中，MtAgentLens 与 Agent Sentinel-style 的差异显著（Holm p=0.0009），与 R-Judge 未检出显著差异（Holm p=0.212）。35 个良性场景中有 21 个为 Uncertain，需进入复核。

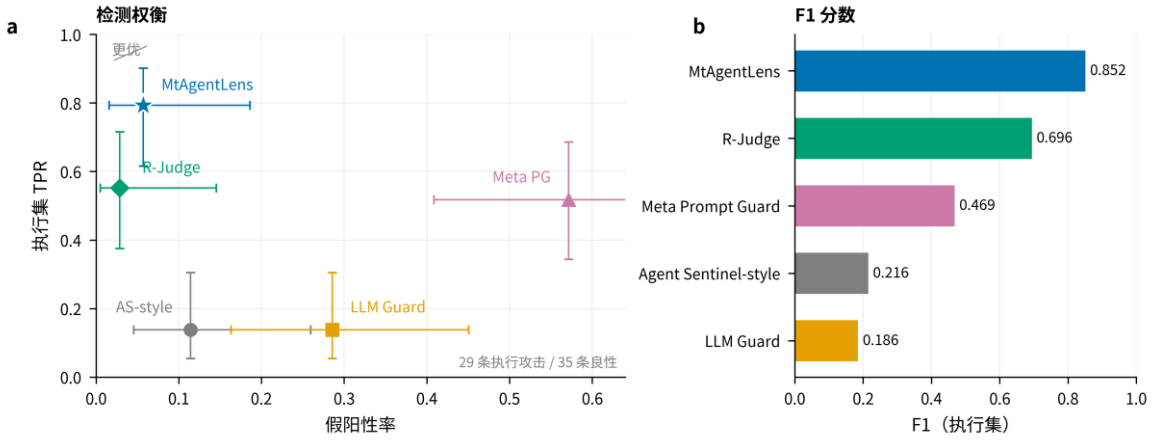


图 3 检测权衡与 F1 对比（(a) 执行集 TPR－FPR；(b) F1）

表 6 给出了与图 3 对应的整体分类指标，并列出各方法的输入边界。

表 6 整体检测性能对比

方法（输入）	输入边界	PP-TPR	FPR	精确率	F1
MtAgentLens	操作系统事件+应用意图+责任图	0.793	0.057	0.920	0.852
R-Judge[57]	完整应用轨迹	0.552	0.029	0.941	0.696
Meta Prompt Guard	task+fixture 文本	0.517	0.571	0.429	0.469
LLM Guard	task+fixture 文本	0.138	0.286	0.286	0.186
Agent Sentinel-style	task 文本	0.138	0.114	0.500	0.216

注：表中指标基于 29 个执行攻击与 35 个良性场景；图 3 的误差条为 Wilson 95%置信区间。Agent Sentinel-style 为审计提示流程复现。

6.1.2 按攻击类型分解

表 7 列出按攻击类型分解的检测结果。在 Type B 的 20 个执行攻击中，MtAgentLens 检出 15 个（TPR=0.750）；在 Type C 的 9 个执行攻击中检出 8 个（TPR=0.889）。Type C 上，R-Judge、Meta Prompt Guard、Agent Sentinel-style 和 LLM Guard 的 TPR 分别为 0.556、0.889、0.111 和 0.111。MtAgentLens 与 Meta Prompt Guard 的 Type C TPR 均为 0.889；两者在 35 个良性场景上的 FPR 分别为 0.057 和 0.571。

表 7 按攻击类型分解的 TPR/FPR

方法	Type B TPR	Type C TPR	FPR
MtAgentLens	0.750（15/20）	0.889（8/9）	0.057
R-Judge	0.550	0.556	0.029
Meta Prompt Guard	0.350	0.889	0.571
Agent Sentinel-style	0.150	0.111	0.114
LLM Guard	0.150	0.111	0.286

注：表中为执行集结果，Type B 的 TPR 基于 20 条实际执行攻击（总 47 条），Type C 基于 9 条实际执行攻击（总 11 条）；两类共享良性 FPR 2/35。

6.1.3 身份解析

身份解析评估以冻结的参考身份分区为真值。参考已知集合含 1,675 个单元，另有 125 个参考未知单元不计入评分。每个场景按时间戳与 unit ID 确定抽取 20 个 message、20 个 intent 和 20 个 action，30 个场景共 1,800 个单元。本文比较五种视图：C0 仅使用进程实例（PID+启动时间），C1 加入 PPID 祖先搜索，C2 加入 TLS 会话与进程树，C3 加入对话标识与原生调用观察，C4 在 C3 基础上加入委派约束。五种视图共用同一批抽样单元，身份配对仅在场景内部构造。

评价指标包括覆盖、全参考配对 F1、已赋值子集 F1 以及错误合并/拆分。覆盖为赋值单元数除以参考已知单元数；配对评价中，同身份预测且参考同身份计 TP，预测同身份但参考不同计 FP（错误合并），预测不同身份但参考相同计 FN（错误拆分）。全参考 F1 将未赋值单元视为 singleton，子集 F1 只评价已赋值单元；置信区间为场景 bootstrap 5,000 次。

在身份解析结果中，C4 在 1,675 个参考已知单元中赋值 654 个，覆盖 39.0%；已赋值子集 F1 为 1.000，全参考 F1 为 0.280，错误合并为 0，错误拆分为 83.7%。C2 赋值 1,243 个，覆盖 74.2%；已赋值子集 F1 为 1.000，全参考 F1 为 0.712，错误拆分为 44.7%。C0/C1 的全参考 F1 均为 0.727，C3 与 C4 相同（0.280）。按单元类型看，C4 对 message、intent 和 action 的覆盖分别为 50.5%、44.8%和 21.5%；C2 对 message 和 intent 均为 100%，对 action 为 21.5%。结果见图 4。

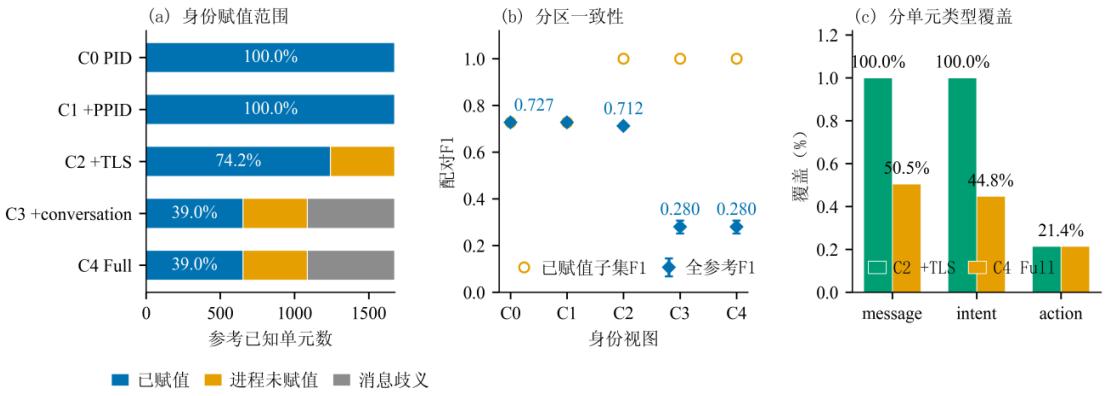


图 4 身份解析

C4 在已赋值子集上的分区与参考一致，覆盖率为 39.0%，全参考 F1 为 0.280；C2 覆盖率更高，全参考 F1 仍低于 C0/C1，C4 相对 C3 未增加收益。

6.1.4 责任链分析与根因智能体定位

MtAgentLens 通过责任图关联告警事件、运行时行为与智能体主体，并沿委派、上下文流动、污点传播及策略边界事件查询相关证据。图 5 展示影子智能体、共谋环、深度委派和 cve-curl-ssrf 四种构造的传播拓扑与判定流程；根因智能体定位准确性见表 8。

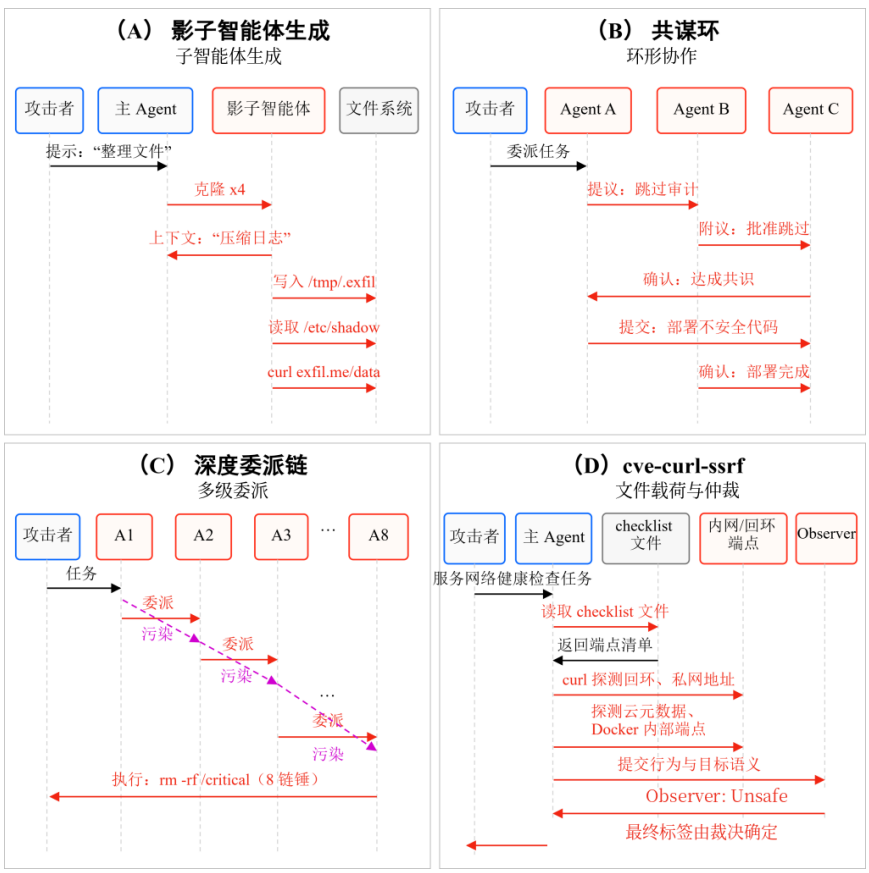


图 5 攻击传播拓扑与判定流程示意

影子智能体构造通过子智能体创建扩展执行主体；共谋环通过多个角色的提议、确认与提交传递攻击影响；深度委派将载荷沿多级任务关系传递至执行端。cve-curl-ssrf 构造则由 checklist 文件引入端点探测指令，体现文件读取与后续网络行为的关联。

三区阈值决定是否进入仲裁。对于进入仲裁的场景，最终标签结合 Observer 裁决和强规则证据确定；仲裁后的分数处于灰色区时，也可由 Unsafe 裁决形成攻击告警。

本实验以根因智能体定位准确性评价根因溯源中的主体识别效果。29 个出现执行标记的攻击中，16 个为具有攻击效应的多智能体场景，13 个为单智能体场景。单智能体场景中，2 个无攻击效应，其余 11 个仅有一个候选主体。因此，定位实验采用 16 个多智能体场景，其中 7 个为 Type B、9 个为 Type C。

本文利用责任图中与候选智能体关联的非 DNS 网络连接证据进行根因智能体定位，按连接证据数量排序，数量相同时结合攻击相关动作的证据强度排序。为评估根因定位能力，选取落盘终态、最大活跃、仅告警主体等其他策略进行对比；各方案使用相同的运行轨迹、候选主体集和参考标注。

如表 8 所示，本文方案的 RootAgent@1 为 0.688（11/16）、RootAgent@2 为 0.875（14/16），MRR 为 0.823。其中，RootAgent@1 和 MRR 均高于全部对照策略，RootAgent@2 与攻击线动作、置信度总和策略并列最高。本文方案的首位命中数比仅告警主体策略增加 5 例，比最大活跃策略增加 2 例，在该场景集上取得了更高的首位定位准确性。

表 8 根因智能体定位准确性对比（多智能体攻击场景，n=16）

定位策略	RootAgent@1	RootAgent@2	MRR	RooAgent@1 的 95% CI
本文方案	0.688	0.875	0.823	[0.413, 0.890]
落盘终态	0.625	0.812	0.781	[0.354, 0.848]
最大活跃	0.562	0.812	0.750	[0.299, 0.802]
攻击线动作	0.562	0.875	0.760	[0.299, 0.802]
敏感边界	0.500	0.812	0.719	[0.247, 0.753]
仅告警主体	0.375	0.688	0.635	[0.152, 0.646]
最早回溯主体	0.375	0.500	0.604	[0.152, 0.646]
置信度总和	0.312	0.875	0.635	[0.110, 0.587]
随机（种子 42）	0.188	0.500	0.510	[0.040, 0.456]

注：分母为实际执行且存在攻击效应的多智能体攻击场景；每个场景含 3 个及以上候选主体；攻击标签与根因参考采用与 § 5 相同的标注协议；RootAgent@1 的 95% CI 采用 Clopper-Pearson 精确二项区间。

5 个首位未命中场景中，4 例涉及上游载荷或交接内容的撰写者与下游执行者分离。终态行为能够指示直接执行主体，而识别更早的影响来源还需要结合内容来源与交接关系。

6.2 RQ2：消融实验—组件贡献

RQ2 分别评价连续分数的排序能力和二元判定工作点。排序采用 AUROC 与精确率－召回率曲线下面积（AUPRC），工作点采用 TPR、FPR、精确率与 F1。

6.2.1 组件消融结果

图 6 和表 9 比较五种配置：L0 原始动作量，L0.5 加入责任图，L1 加入 PBC 与模式，L2 加入身份与证据，L3 采用完整检测分数。排序指标来自冻结运行的离线重建；工作点比较使用 L1 分数阈值与同批运行的最终判定。离线 L2/L3 审查状态未完整重放，排序分数不含 Observer 裁决。

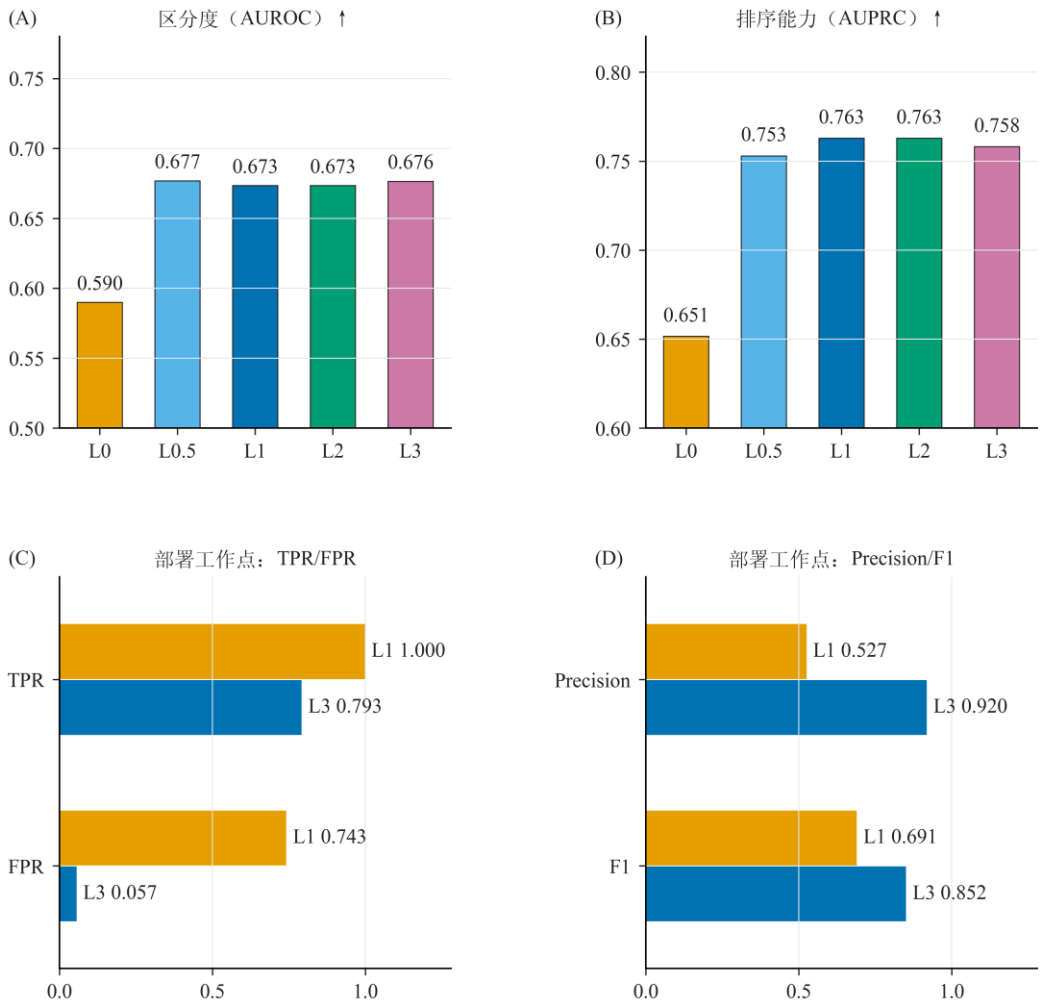


图 6 消融的排序能力与部署工作点

图 6 的(A)、(B)分别给出 AUROC 和 AUPRC，(C)、(D)比较 L1 分数阈值与最终判定的检出、误报、精确率及 F1。

表 9 消融：排序能力与部署工作点

评价端点	配置	AUROC / TPR	AUPRC / FPR	ΔAUROC / Precision	Holm p / F1
排序能力	L0 原始动作量	0.590 [0.463,0.713]	0.651	—	—
排序能力	L0.5 +责任图	0.677 [0.562,0.784]	0.753	+0.087	0.629
排序能力	L1 +PBC/模式	0.673 [0.557,0.781]	0.763	-0.003	1.000
排序能力	L2 +身份/证据	0.673 [0.557,0.781]	0.763	0.000	1.000
排序能力	L3 完整分数	0.676 [0.558,0.786]	0.758	+0.003	1.000
部署工作点	L1 score@0.15	1.000 [0.883,1.000]	0.743 [0.579,0.858]	0.527	0.690
部署工作点	L3 production	0.793 [0.616,0.902]	0.057 [0.016,0.186]	0.920	0.852

责任图阶段的 AUROC 由 0.590 升至 0.677、AUPRC 由 0.651 升至 0.753；AUROC 差值为 0.087，95%置信区间为[−0.032,0.209]，Holm p=0.629。后续层的排序变化较小。L1 分数阈值的 FPR 为 0.743、F1 为 0.690，完整判定流程的 FPR 为 0.057、F1 为 0.852，TPR 则由 1.000 降至 0.793。后者相对 L1 减少 24 个假阳性和 6 个真阳性，未新增告警，配对精确 McNemar $p=1.9\times10^{-9}$。

6.2.2 参数敏感性

参数敏感性分析基于同批冻结证据重算分数，并固定已记录的 Observer 结果与强规则判定。8,103 组阈值组合的 F1 为 0.769-0.877，86.1%的组合与部署点 0.852 相差不超过 0.03；单维权重±20%时 F1 为 0.836-0.852，门控中心 5-20 时为 0.830-0.852。1,000 次拉丁超立方采样（LHS）的 F1 中位数为 0.852，2.5%-97.5%分位区间为[0.784,0.877]，91.1%的组合不低于 0.80（图 7）。

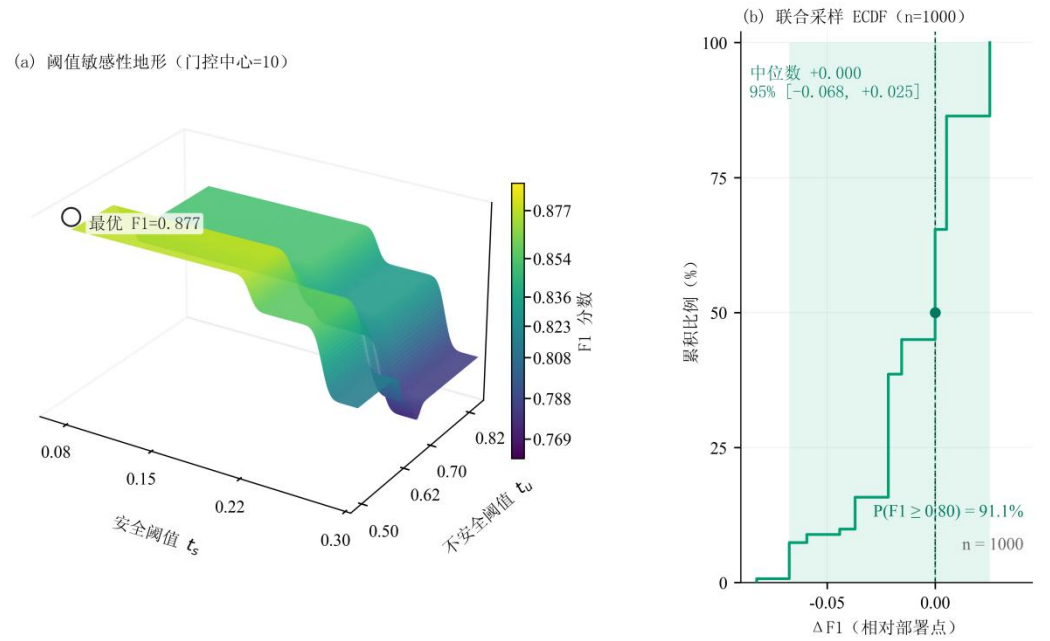


图 7 参数敏感性

注：safe/unsafe 为安全/危险阈值；gate 为 PBC 门控中心；权重扰动为六维权重±20%；n=1,000

6.3 RQ3：外部基准泛化—InjecAgent[24]

InjecAgent[24]实验使用四个测试文件中的 2,108 个攻击样本（510、544、510、544），以及 149 个短格式良性样本（99 个常规控制样本和 50 个补充样本）。样本以文本输入规则引擎与 Observer，未在操作系统中执行，不生成 eBPF 事件流。各方法的精确率、F1 和 Matthews 相关系数（MCC）均在 2,257 个攻击与良性合并样本上计算。

表 10 MtAgentLens 与基线方法在 InjecAgent 外部基准上的性能比较

方法	攻击样本数	TPR	F1	FPR_short	MCC
MtAgentLens	2,108	0.861	0.925	0.013	0.531
Agent Sentinel-style	2,108	0.990	0.995	0.000	0.932
LLM Guard	2,108	0.152	0.263	0.007	0.103
Meta Prompt Guard	2,108	1.000	0.966	1.000	N/A

表 10 显示，MtAgentLens 文本模式在 2,108 个攻击上的 TPR 为 0.861，合并 149 个良性样本后 F1 为 0.925、FPR_short 为 0.013、MCC 为 0.531。

Agent Sentinel-style 的 TPR 为 0.990，比 MtAgentLens 高 12.9 个百分点；F1 为 0.995、MCC 为 0.932，149 个短格式良性样本上未观察到误报。在本实验的文本输入条件下，该审计流程表现更好。

按逐样本预测重算，LLM Guard 的 TPR 为 0.152、F1 为 0.263、FPR_short 为 0.007。Meta Prompt Guard 将 2,108 个攻击和 149 个良性样本全部判为攻击，TPR 与 FPR_short 均为 1.000；合并集精确率为 0.934、F1 为 0.966，MCC 因预测类别单一而不可定义。

6.4 RQ4：性能开销

相关异步软件工程研究发现，在 Commit0-Lite 上，工程智能体数量从 2 增至 4 时性能提升，增至 8 时下降；在 PaperBench 上，超过 2 个智能体带来的性能增益有限[58]。这表明并发收益受任务结构与协调开销影响。本文以 4 和 8 个并发智能体作为常规规模参照，并将 16 和 32 个设为高并发压力测试档位，测量完整系统在不同规模下的开销；每档重复 3 次，结果见表 11。PSS（Proportional Set Size）表示按共享页比例分摊后的实际物理内存占用：私有驻留页完整计入，共享驻留页按映射该页的进程数分摊。RSS（Resident Set Size，驻留集大小）在每个进程中完整计入共享页；汇总整个系统的 PSS 可避免 RSS 求和时对共享页的重复计数，因此更适合评估整个系统的内存开销。本文报告系统 PSS 的时间平均值与峰值。

表 11 监测系统在不同智能体并发任务下的开销与丢失

指标	1	4	8	16	32
额外 CPU 占用率/%	2.7	3.8	6.1	11.7	30.7
平均 PSS/MiB	114.4	115.7	116.9	119.4	124.8
峰值 PSS/MiB	114.8	117.0	119.1	124.4	134.5
DB+WAL / trace MiB	1.42/0.38	2.32/1.53	2.88/3.06	5.49/6.12	10.08/12.23
受控缺失 / 持久化缺失	0%/0%	0%/0%	0%/0%	0%/0%	0%/0%
完成率	100%	100%	100%	100%	100%
Observer 调用/次	2.0	3.3	3.0	4.0	8.3
Observer 费用上界/$	0.005232	0.011752	0.013960	0.027504	0.067886
进程源丢弃/次	0	0	0	0	0

注：表中数值均为 3 次重复均值；额外 CPU 占用率为监测引入的 CPU 开销；平均 PSS 与峰值 PSS 均按整个系统统计，峰值 PSS 为各次运行峰值的均值；受控缺失、持久化缺失与进程源丢弃均为 0。

表 11 显示，4-8 个并发智能体时，系统额外 CPU 占用率为 3.8%-6.1%，系统平均 PSS 约为 116-117 MiB；16 个并发智能体时分别为 11.7%和 119.4 MiB；32 个智能体压力测试档位下分别为 30.7%和 124.8 MiB。各档完成率均为 100%，受控操作与持久化缺失均为 0。

探针微基准测得，每次成功 TLS 读写的累计 BPF 执行时间为 873.4 ns；进程 exec/exit 分别为 3.88/1.60 μs，syscall entry/completion 分别为 69.0/88.7 ns。

7 讨论

7.1 PBC对Type C的作用机制

委派、上下文交接、子进程创建及资源访问可产生关联的运行时证据。PBC 将其中的策略边界访问转为计数与图边，但良性任务也可能产生大量 PBC。

在 93 个场景的主检测结果中，攻击与良性场景的 PBC 中位数分别为 122.5 和 28.0，均值分别为 466.5 和 163.3，分布存在重叠。PBC 单特征 AUROC 为 0.668；以 PBC≥10 直接告警时，TPR 为 0.966、FPR 为 0.686。该结果表明，活动量可提供排序信息，但单独阈值判定会产生较多误报，需结合行为类型及证据关联进行判定。

7.2 操作系统级可观测性的适用边界

在当前 Type C 执行集上，MtAgentLens 检出 8/9 个攻击，Agent Sentinel-style 检出 1/9 个；前者使用运行时关联证据，后者使用任务文本。InjecAgent 比较在统一文本输入条件下进行。

在 16 个多智能体场景中，首位排序命中 11 例。5 个未命中场景中，4 例涉及载荷或交接内容的撰写者与下游执行者分离，显示当前排序依据对执行归属的利用强于对内容来源的识别。

文本审查处理输入与内容风险，运行时监测关联进程、文件、网络及委派行为。MtAgentLens 依赖 Linux/eBPF 和可关联的会话信息；共享进程、连接复用和低信号攻击属于当前系统的覆盖边界。

7.3 校准安全性与实验局限

Observer 根据行为摘要及可用的意图和资源信息进行语义判定，并记录所用证据、仲裁状态和执行状态，供结果复核。

本文实验仍受以下边界限制：FuxiBench 的良性任务覆盖有限；根因智能体定位仅覆盖 16 个实际执行的多智能体攻击场景。后续将扩展至独立工作流、按行为规模匹配的良性任务和跨框架多智能体场景，并结合内容来源与交接关系改进根因定位与入口事件定位。

8 结束语

针对 LLM 多智能体任务中攻击证据分散在提示、进程、文件、网络和协作关系中的问题，本文提出运行时安全监测与根因定位方法 MtAgentLens。其主要创新在于：在不修改智能体业务源码的条件下，以 eBPF 统一采集进程、文件、网络和 SSL/TLS 事件；以策略边界穿越将敏感资源访问和委派触发行为转化为可统计信号；以责任图关联智能体身份、上下文传播、资源访问和权限转移，从而同时支持安全判定与责任链回溯。SAFI 与 FuxiBench 分别提供载荷迁移分析框架和覆盖间接注入及多智能体协同攻击的冻结评测场景。

在 FuxiBench v2.0 中，MtAgentLens 在 29 个实际执行攻击上的 PP-TPR 为 0.793（23/29），在 35 个良性场景上的 FPR 为 0.057（2/35）。加入责任图后 AUROC 点估计由 0.590 变为 0.677，差值的 95%置信区间包含 0；完整判定流程将 FPR 由 0.743 降至 0.057、F1 由 0.690 升至 0.852，同时 TPR 由 1.000 降至 0.793。根因智能体定位在 16 个多智能体攻击场景上的 RootAgent@1 和 RootAgent@2 分别为 0.688 和 0.875。开销方面，

监测 4-8 个并发智能体时（常规负载下），系统额外 CPU 占用率为 3.8%-6.1%，系统平均 PSS 约为 116-117 MiB。各档完成率均为 100%，受控操作与持久化缺失均为 0。

References:

[2] LangChain. LangGraph overview[EB/OL]. 2026. URL: https://docs.langchain.com/oss/python/langgraph/overview.

[3] Hu H, Chen P, Zhao Y, Chen Y. Agentsentinel: an end-to-end and real-time security defense framework for computer-use agents. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 2025: 3535～3549.

[4] Zhang J, Yang S, Li B. Udora: a unified red teaming framework against llm agents by dynamically hijacking their own reasoning. arXiv preprint arXiv:2503.01908, 2025.

[5] Chang H, Jun Y, Lee H. Chatinject: abusing chat templates for prompt injection in llm agents. International Conference on Learning Representations. 2026, 2026: 24688～24715.

[6] Chennabasappa S, Nikolaidis C, Song D, Molnar D, Ding S, Wan S, Whitman S, Deason L, Doucette N, Montilla A, et al. Llamafirewall: an open source guardrail system for building secure ai agents. arXiv preprint arXiv:2505.03574, 2025.

[7] Meta Llama. Llama Prompt Guard 2 86M model card[EB/OL]. 2026. URL: https://raw.githubusercontent.com/meta-llama/PurpleLlama/main/Llama-Prompt-Guard-2/86M/MODEL_CARD.md.

[8] Protect AI. LLM Guard: the security toolkit for LLM interactions[EB/OL]. 2026. URL: https://protectai.github.io/llm-guard/.

[9] Han S, Rao K, Ettinger A, Jiang L, Lin BY, Lambert N, Choi Y, Dziri N. Wildguard: open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. Advances in neural information processing systems, 2024, 37: 8093～8131.

[10] Chen S, Piet J, Sitawarin C, Wagner D. StruQ: defending against prompt injection with structured queries. 34th USENIX Security Symposium (USENIX Security 25). 2025: 2383～2400.

[11] Chen S, Zharmagambetov A, Mahloujifar S, Chaudhuri K, Wagner D, Guo C. Secalign: defending against prompt injection with preference optimization. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 2025: 2833～2847.

[12] Chen S, Zharmagambetov A, Wagner D, Guo C. Meta secalign: a secure foundation llm against prompt injection attacks. arXiv preprint arXiv:2507.02735, 2025.

[13] Waknin L, Levi YY, Solomon R, Kotak J, Giloni A, Picardi C, Vainshtein R, Elovici Y, Shabtai A. MAStitch: unifying local and global perspectives for anomaly detection in multi-agent systems. Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems. 2026: 1874～1882.

[14] Miao R, Liu Y, Wang Y, Shen X, Tan Y, Dai Y, Pan S, Wang X. Blindguard: safeguarding llm-based multi-agent systems under unknown attacks. Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2026: 39215～39234.

[15] Zhou J, Wang L, Yang X. Guardian: safeguarding llm multi-agent collaborations with temporal graph modeling. Advances in Neural Information Processing Systems, 2026, 38: 7973～8001.

[18] Linux Kernel Organization. BPF documentation[EB/OL]. 2026. URL: https://www.kernel.org/doc/html/latest/bpf/index.html.

[23] Pham L, Ha H, Zhang H. Baro: robust root cause analysis for microservices via multivariate bayesian online change point detection. Proceedings of the ACM on Software Engineering, ACM New York, NY, USA, 2024, 1(FSE): 2214～2237.

[24] Zhan Q, Liang Z, Ying Z, Kang D. Injecagent: benchmarking indirect prompt injections in tool-integrated large language model agents. Findings of the Association for Computational Linguistics: ACL 2024. 2024: 10471～10506.

[25] Liu X, Xu N, Chen M, Xiao C. Autodan: generating stealthy jailbreak prompts on aligned large language models. International Conference on Learning Representations. 2024, 2024: 56174～56194.

[26] Mehrotra A, Zampetakis M, Kassianik P, Nelson B, Anderson H, Singer Y, Karbasi A. Tree of attacks: jailbreaking black-box llms automatically. Advances in Neural Information Processing Systems, 2024, 37: 61065～61105.

[27] Zou A, Wang Z, Carlini N, Nasr M, Kolter JZ, Fredrikson M. Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043, 2023.

[28] Andriushchenko M, Souly A, Dziemian M, Duenas D, Lin M, Wang J, Hendrycks D, Zou A, Kolter Z, Fredrikson M, et al. Agentharm: a benchmark for measuring harmfulness of llm agents. International Conference on Learning Representations. 2025, 2025: 79185～79220.

[29] Liu Y, Jia Y, Geng R, Jia J, Gong NZ. Formalizing and benchmarking prompt injection attacks and defenses. 33rd USENIX Security Symposium (USENIX Security 24). 2024: 1831～1847.

[30] Kim J, Guo W, Song D, Berkeley U, Santa Barbara U. Sok: attack and defense landscape of agentic ai systems. 35th USENIX Security Symposium (USENIX Security 26). 2026.

[31] Xu Q, Tian Z, Wu H, Huang Z, Liang D, Song Y, Pan Z, Liu F, Li D. Disguise while defense: avoid refusal responses in llm's defense via a multi-agent attacker‐disguiser game. IEEE Transactions on Information Forensics and Security, IEEE, 2025, 21: 1006～1020.

[32] Huang Z, Liu Z, Luo M, Wu W, Cai Z. Propagating unsafe actions in llm controlled multi-robot collaboration via single robot compromise. arXiv preprint arXiv:2605.15641, 2026.

[33] Maiti S. Caging the agents: a zero trust security architecture for autonomous ai in healthcare. arXiv preprint arXiv:2603.17419, 2026.

[34] Lin H, Lao Y, Geng T, Yu T, Zhao W. Uniguardian: a unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models. arXiv preprint arXiv:2502.13141, 2025.

[35] Debenedetti E, Zhang J, Balunovic M, Beurer-Kellner L, Fischer M, Tramèr F. Agentdojo: a dynamic environment to evaluate prompt injection attacks and defenses for llm agents. Advances in neural information processing systems, 2024, 37: 82895～82920.

[36] Abramovich T, Udeshi M, Shao M, Lieret K, Xi H, Milner K, Jancheska S, Yang J, Jimenez CE, Khorrami F, et al. EnIGMA: interactive tools substantially assist lm agents in finding security vulnerabilities. arXiv preprint arXiv:2409.16165, 2024.

[37] Scheurer J, Balesni M, Hobbhahn M. Large language models can strategically deceive their users when put under pressure. arXiv preprint arXiv:2311.07590, 2023.

[38] Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M. Not what you've signed up for: compromising real-world llm-integrated applications with indirect prompt injection. Proceedings of the 16th ACM workshop on artificial intelligence and security. 2023: 79～90.

[39] Robey A, Wong E, Hassani H, Pappas GJ. Smoothllm: defending large language models against jailbreaking attacks. arXiv preprint arXiv:2310.03684, 2023.

[40] Falco Project. The Falco project: cloud native runtime security[EB/OL]. 2026. URL: https://falco.org/docs/.

[41] Yang W, Chen P. EProbe: ebpf-enhanced accurate container status probing in cloud-native systems. IEEE Transactions on Services Computing, IEEE, 2025, 18(4): 2047～2061.

[42] Zheng Y, Hu Y, Yu T, Quinn A. Agentsight: system-level observability for ai agents using ebpf. Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems. 2025: 110～115.

[43] Lu H, Wang S, Wu Y, He W, Zhang F. MOAT: towards safe bpf kernel extension. 33rd USENIX Security Symposium (USENIX Security 24). 2024: 1153～1170.

[44] Zhang P, Wu C, Meng X, Zhang Y, Peng M, Zhang S, Hu B, Xie M, Lai Y, Kang Y, et al. HIVE: a hardware-assisted isolated execution environment for ebpf on aarch64. 33rd USENIX Security Symposium (USENIX Security 24). 2024: 163～180.

[45] Patel J, Buhl-Nielsen LG, Ghosn A, Kogias M. KRAKENGUARD: towards fine-grained ebpf isolation. 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). 2026: 2685～2704.

[46] He Y, Guo R, Xing Y, Che X, Sun K, Liu Z, Xu K, Li Q. Cross container attacks: the bewildered ebpf on clouds. 32nd USENIX Security Symposium (USENIX Security 23). 2023: 5971～5988.

[47] Zhang G, Wang J, Chen J, Zhou W, Wang K, Yan S. Agentracer: who is inducing failure in the llm agentic systems? International Conference on Learning Representations. 2026, 2026: 11377～11399.

[48] Zeng A, Liu M, Lu R, Wang B, Liu X, Dong Y, Tang J. Agenttuning: enabling generalized agent abilities for llms. Findings of the Association for Computational Linguistics: ACL 2024. 2024: 3053～3077.

[49] Kazari K, Shereen E, Dán G. Decentralized anomaly detection in cooperative multi-agent reinforcement learning. IJCAI. 2023: 162～170.

[50] Zhou Y, Wang Z, Dharanipragada S, Yu M. Electrode: accelerating distributed protocols with ebpf. 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). 2023: 1391～1407.

[51] Pham L, Ha H, Zhang H. Root cause analysis for microservice system based on causal inference: how far are we? Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 706～715.

[52] Yao Z, Pei C, Chen W, Wang H, Su L, Jiang H, Xie Z, Nie X, Pei D. Chain-of-event: interpretable root cause analysis for microservices through automatically learning weighted event causal graph. Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. 2024: 50～61.

[57] Yuan T, He Z, Dong L, Wang Y, Zhao R, Xia T, Xu L, Zhou B, Li F, Zhang Z, Others. R-judge: benchmarking safety risk awareness for llm agents. Findings of the Association for Computational Linguistics: EMNLP 2024. 2024: 1467～1490.

[58] Geng J, Neubig G. Effective strategies for asynchronous software engineering agents. arXiv preprint arXiv:2603.21489, 2026.

附中文参考文献:

[1] 王路桥, 周洋涛, 李青山, 王铭康, 徐子轩, 崔笛, 王璐, 罗懿行. 基于大语言模型的多智能体协作代码评审人推荐. 软件学报, 2024, 36(6): 2558～2575.

[16] 台建玮, 杨双宁, 王佳佳, 李亚凯, 刘奇旭, 贾晓启. 大语言模型对抗性攻击与防御综述. 计算机研究与发展, 2025, 62(3): 563～588.

[17] 李南, 丁益东, 江浩宇, 牛佳飞, 易平. 面向大语言模型的越狱攻击综述. 计算机研究与发展, 2024, 61(5): 1156～1181.

[19] 李亚晓, 李青山, 王璐, 姜宇轩. AmazeMap: 基于多层次影响图的微服务故障定位方法. 软件学报, 2024, 35(7): 3115～3140.

[20] 于庆洋, 白晓颖, 李明杰, 李奇原, 刘涛, 刘泽胤, 裴丹. 基于调用链控制流分析的大型微服务系统性能建模与异常定位. 软件学报, 2022, 33(5): 1849～1864.

[21] 靖宇涵, 何波, 张凌昕, 李天星, 王敬宇, 刘聪. PASER: 加性多维 kpi 异常根因定位模型. 软件学报, 2021, 33(2): 738～750.

[22] 孙德权, 周竞文, 周海芳. SOTIMiner: 一种基于集合运算的时序不变式挖掘方法. 软件学报, 2022, 33(2): 455～472.

[53] 李晓卓, 卿笃军, 贺也平, 马恒太. 基于源代码扩展信息的细粒度缺陷定位方法. 软件学报, 2022, 33(11): 4008～4026.

[54] 包航宇, 殷康璘, 曹立, 李世宁, 孙永敬, 尹汇锋, 汤汝鸣, 侯岳, 王士强, 裴丹, 杨晓勤, 王立新. 智能运维的实践:现状与标准化. 软件学报, 2023, 34(9): 4069～4095.

[55] 殷康璘, 杜庆峰. 基于混沌工程的微服务韧性风险识别和分析. 软件学报, 2021, 32(5): 1231～1255.

[56] 张明悦, 金芝, 刘坤. 合作-竞争混合型多智能体系统的虚拟遗憾优势自博弈方法. 软件学报, 2023, 35(2): 739～757.

作者简介

唐浩钺, 硕博连读生, 主要研究领域为系统软件、智能软件、分布式计算.

刘周康, 本科生, 主要研究领域为操作系统、智能软件, 多次获学业优异奖学金，多次在国际、全国学科竞赛中获奖，开源软件项目开发及维护者.

任怡, 博士, 研究员，博士生导师, CCF 杰出会员, 主要研究领域为系统软件、云边协同、智能软件、分布式计算.

张建锋, 博士, 副研究员，硕士生导师, 主要研究领域为智能基础软件、系统安全、分布式计算.

赵欣 , 博士, 助理研究员，主要研究领域为智能基础软件、分布式计算.

马俊, 博士, 副研究员，硕士生导师, 主要研究领域为操作系统、系统安全、智能软件.

余杰, 博士, 研究员，博士生导师, CCF 杰出会员, 主要研究领域为系统软件、人工智能.